

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA TOÁN - TIN HỌC



Khóa luận tốt nghiệp

CẢI THIẾN THUẬT TOÁN SELF-DISTILLATION
POLICY OPTIMIZATION

CHO MÔ HÌNH NGÔN NGỮ LỚN
TRONG BÀI TOÁN SUY LUẬN TOÁN HỌC

Tp. Hồ Chí Minh - 2026

**ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA TOÁN - TIN HỌC**



Khóa luận tốt nghiệp

**CẢI THIẾN THUẬT TOÁN SELF-DISTILLATION
POLICY OPTIMIZATION**

**CHO MÔ HÌNH NGÔN NGỮ LỚN
TRONG BÀI TOÁN Suy Luận Toán Học**

CHUYÊN NGÀNH KHOA HỌC DỮ LIỆU

Giảng viên hướng dẫn: TS. NGÔ MINH Mẫn

Sinh viên thực hiện: TÔ GIA BẢO - 22280006

PHAN VĂN HOÀNG - 22280033

Tp. Hồ Chí Minh - 2026

LỜI CẢM ƠN

Lời đầu tiên, chúng em xin gửi lời cảm ơn chân thành đến Khoa Toán - Tin học và Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM đã tạo điều kiện cho chúng em được học tập và rèn luyện trong môi trường giáo dục chất lượng cao, giúp chúng em có cơ hội thực hiện khóa luận tốt nghiệp này.

Chúng em đặc biệt biết ơn TS. Ngô Minh Mẫn, người đã tận tâm hướng dẫn, chỉ bảo chúng em trong suốt quá trình thực hiện đề tài. Nhờ sự nhiệt tình và tận tâm của thầy, chúng em đã được trang bị những kiến thức và kỹ năng cần thiết để hoàn thành khóa luận tốt nghiệp một cách tốt đẹp nhất.

Chúng em cũng xin gửi lời tri ân đến gia đình, đặc biệt là ba mẹ - những người đã luôn yêu thương, động viên và tạo điều kiện tốt nhất cho chúng em học tập và theo đuổi đam mê. Nhờ sự ủng hộ tinh thần to lớn của gia đình, chúng em đã có thêm động lực.

LỜI MỞ ĐẦU

Trong những năm gần đây, các Mô hình Ngôn ngữ Lớn (LLMs) đã đạt được những bước tiến vượt bậc, thể hiện khả năng xuất sắc trong nhiều lĩnh vực, đặc biệt là các tác vụ đòi hỏi tư duy logic như suy luận toán học. Tuy nhiên, để các mô hình này thực sự làm chủ được các bài toán nhiều bước, quá trình tinh chỉnh (post-training) bằng học tăng cường đóng vai trò vô cùng quan trọng.

Các phương pháp học tăng cường truyền thống, chẳng hạn như GRPO, thường chỉ tối ưu hóa dựa trên một phần thưởng vô hướng duy nhất ở cuối chuỗi sinh văn bản. Đối với các bài toán toán học phức tạp, điều này tạo ra "nút thắt phân bổ tín nhiệm" nghiêm trọng, vì mô hình không thể nhận biết chính xác bước lập luận nào trong toàn bộ chuỗi tư duy là sai lầm. Gần đây, thuật toán Self-Distillation Policy Optimization (SDPO) đã ra đời nhằm biến các phản hồi văn bản thành tín hiệu tự sửa lỗi. Dù mang lại hiệu quả đột phá, SDPO khi áp dụng vào suy luận toán học vẫn đối mặt với bài toán học búa về độ tin cậy của quá trình tự học và chi phí tính toán cao.

Xuất phát từ thực tiễn trên, đề tài: **“Cải thiện thuật toán Self-Distillation Policy Optimization cho mô hình ngôn ngữ lớn trong bài toán suy luận toán học”** được lựa chọn để nghiên cứu. Khóa luận tập trung triển khai cơ chế Cổng độ tin cậy (Reliability Gate) nhằm lọc tín hiệu nhiễu trong quá trình tự chưng cất; hệ thống được xây dựng và huấn luyện trên dòng mô hình Qwen3 thông qua nền tảng mã nguồn mở Verl.

Kết quả thực nghiệm cho thấy cơ chế cải tiến đã giúp mô hình chọn lọc hiệu quả các chuỗi tư duy chất lượng và tránh việc học theo các suy luận sai lệch (misleading reasoning). Hơn nữa, thông qua cơ chế thực thi thưa (sparse target execution), thuật toán loại bỏ hoàn toàn việc tính toán hàm mất mát cho các mẫu rác, qua đó tối ưu hóa mạnh mẽ tài nguyên phần cứng. Việc áp dụng thành công SDPO cải tiến không chỉ nâng cao tính ổn định, tiết kiệm chi phí huấn luyện mà còn cải thiện rõ rệt độ chính xác của

LLMs khi giải quyết các bài toán suy luận phức tạp.

DANH SÁCH TỪ VIẾT TẮT

(Phần nội dung danh sách từ viết tắt - Sinh viên tự bổ sung sau)

Mục lục

LỜI CẢM ƠN	1
LỜI MỞ ĐẦU	2
DANH SÁCH TỪ VIẾT TẮT	4
1 GIỚI THIỆU ĐỀ TÀI	1
1.1 Bối cảnh của đề tài	1
1.2 Lý do thực hiện đề tài, vấn đề nghiên cứu	1
1.3 Mục tiêu nghiên cứu	2
1.4 Phương pháp nghiên cứu	3
1.5 Kết cấu của đề tài và giới hạn đề tài	3
2 CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ TƯƠNG QUAN	5
2.1 Tổng quan về Học tăng cường (Reinforcement Learning)	5
2.1.1 Chính sách và Chính sách tối ưu (Policy & Optimal Policy)	6
2.1.2 Hàm giá trị (Value Function)	7
2.2 Mô hình Ngôn ngữ Lớn và Suy luận Toán học	7
2.3 Học tăng cường: Từ GRPO đến SDPO	8
2.3.1 Học tăng cường với phần thưởng kiểm chứng (RLVR) và Phản hồi phong phú (RLRF)	8

2.3.2	Tối ưu hóa Chính sách Khen thưởng Nhóm (GRPO)	10
2.3.3	Tối ưu hóa Chính sách Tự chưng cất (SDPO)	11
2.4	Biểu đạt nhận thức (Epistemic Verbalization) và Rủi ro triệt tiêu	14
2.4.1	Khái niệm Biểu đạt nhận thức trong suy luận Toán học	15
2.4.2	Lượng tin tương hỗ và Sự thay đổi hành vi suy luận	16
2.4.3	Hiện tượng Triệt tiêu nhận thức (Epistemic Suppression)	16
2.5	Các công nghệ và nền tảng hỗ trợ	17
3	PHƯƠNG PHÁP ĐỀ XUẤT VÀ THIẾT KẾ	20
3.1	Giới hạn của SDPO truyền thống trong lĩnh vực Toán học	20
3.2	Đề xuất SDPO kết hợp Cổng độ tin cậy (Reliability-Gated SDPO)	21
3.2.1	Hàm mục tiêu (Objective Function)	21
3.2.2	Cơ chế gán trọng số tín nhiệm (Reliability Weights)	22
3.2.3	Cổng lọc và Ngân sách tính toán (Gate & Batch Budget)	22
3.3	Các khía cạnh cải tiến đột phá	23
3.3.1	Nâng cao chất lượng mục tiêu (Higher-quality SDPO targets)	23
3.3.2	Giảm thiểu chi phí tính toán (Lower SDPO compute)	23
4	THỰC NGHIỆM VÀ ĐÁNH GIÁ	25
4.1	Môi trường và dữ liệu thực nghiệm	25
4.2	Thiết lập tham số huấn luyện	25
4.3	Kết quả thực nghiệm	25
4.4	Phân tích và đánh giá kết quả	25
5	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	26
5.1	Kết luận	26
5.2	Những đóng góp chính	26
5.3	Hướng phát triển tiếp theo	26
	TÀI LIỆU THAM KHẢO	27

Danh sách hình vẽ

2.1	Mô hình hóa Quá trình ra quyết định Markov (MDP)	6
2.2	So sánh thiết lập RLVR và RLRF. Trong RLVR, tác tử học từ một phần thưởng vô hướng r , đóng vai trò như một nút thắt thông tin. Ngược lại, RLRF sử dụng các phản hồi dạng token, cung cấp tín hiệu dồi dào giải thích tại sao một nỗ lực lại thất bại.	9
2.3	Mình họa sự khác biệt kiến trúc giữa PPO (đòi hỏi Value Network) và GRPO (loại bỏ hoàn toàn Value Network thông qua lấy mẫu nhóm).	10
2.4	Mình họa sự khác biệt trong việc gán tín nhiệm giữa GRPO và SDPO đối với một bài toán suy luận Toán học. GRPO phạt toàn bộ chuỗi khi kết quả cuối cùng sai, trong khi cơ chế tự chưng cất của SDPO giúp phát hiện chính xác token gây ra lỗi logic (kích hoạt thừa thớt) và bỏ qua các phần suy luận hợp lệ theo sau.	15
2.5	Kiến trúc phân tán của Verl dựa trên nền tảng Ray.	18
2.6	Cơ chế cấp phát bộ nhớ động PagedAttention của vLLM giúp tăng tốc độ sinh dữ liệu.	19

Danh sách bảng

2.1	Khuôn mẫu (Template) dành cho "teacher". prompt được thay bằng câu hỏi. successful_previous_rollout là lời giải mẫu chính xác được sinh ra trong cùng nhóm (sẽ bị bỏ qua đoạn này nếu không có). environment_output là phản hồi lỗi từ môi trường đối với nỗ lực giải ban đầu (bỏ qua nếu đã có lời giải mẫu). Nếu nỗ lực ban đầu là đúng, nó sẽ được chuyển thành lời giải mẫu. Cuối cùng, original_response được thay bằng câu trả lời gốc của mô hình để tính toán lại xác suất (log-probs) dưới lăng kính của teacher.	14
-----	--	----

3.1	Bảng phân bổ Trọng số tín nhiệm cho các mục tiêu SDPO.	23
-----	--	----

Chương 1

GIỚI THIỆU ĐỀ TÀI

1.1 Bối cảnh của đề tài

Sự bùng nổ của các Mô hình Ngôn ngữ Lớn (LLMs) như GPT-4 hay Qwen đã mở ra tiềm năng tự động hóa các tác vụ đòi hỏi tư duy logic phức tạp, tiêu biểu là suy luận toán học. Khác với các tác vụ xử lý ngôn ngữ tự nhiên cơ bản, toán học yêu cầu mô hình phải lập luận qua nhiều bước (Chain-of-Thought) với tính chính xác tuyệt đối.

Tuy nhiên, trí thông minh này không chỉ đến từ việc huấn luyện trước (pre-training) trên lượng dữ liệu khổng lồ. Để LLMs thực sự làm chủ cách giải bài toán theo từng bước lô-gic, quá trình tinh chỉnh (post-training) bằng Học tăng cường (Reinforcement Learning - RL) đóng vai trò quyết định. RL giúp căn chỉnh phản hồi của mô hình theo đúng lô-gic yêu cầu, là chìa khóa để khai mở tiềm năng giải toán của LLMs.

1.2 Lý do thực hiện đề tài, vấn đề nghiên cứu

Mặc dù đóng vai trò then chốt, các thuật toán RL truyền thống (tiêu biểu như GRPO) lại có giới hạn bản chất trong miền suy luận toán học.

Thứ nhất, nút thắt phân bổ tín nhiệm. RL truyền thống thường đánh giá mô hình bằng một điểm số vô hướng duy nhất ở cuối chuỗi sinh (ví dụ: +1 nếu đúng, 0 nếu sai). Khi bài toán cần hàng chục bước giải, việc chỉ cung cấp một điểm số tạo ra "nút thắt phân bổ tín nhiệm" nghiêm trọng. Mô hình không thể nhận biết chính xác bước lập luận nào trong chuỗi là sai lầm.

Thứ hai, lãng phí phản hồi văn bản. Các môi trường kiểm chứng thực tế không

chỉ trả về đúng/sai mà thường cung cấp lỗi hoặc giải thích chi tiết. Các phương pháp RL truyền thống hoàn toàn bỏ qua nguồn dữ liệu phong phú này.

Để khắc phục, Tối ưu hóa Chính sách Tự chưng cất (SDPO) đã được đề xuất. SDPO trực tiếp sử dụng LLM làm "người thầy". Khi giải sai, mô hình nhận phản hồi văn bản, tự sinh lời giải mới tốt hơn, rồi dùng nó để tự chưng cất và cập nhật chính sách.

Vấn đề nghiên cứu đặt ra: Tuy mang tính đột phá, SDPO đối mặt với hai thách thức chí mạng khi áp dụng vào miền Toán học:

- *Mô phỏng suy luận sai lệch (Noisy Imitation):* SDPO nguyên bản giả định mọi mục tiêu chưng cất từ phản hồi đều có ích. Tuy nhiên, trong toán học, một lời giải sai vẫn có thể là một chuỗi suy luận dài, trôi chảy nhưng chứa các bước lập luận sai lầm. Việc "bắt chước" mù quáng các mục tiêu này sẽ làm củng cố các tư duy sai lệch của mô hình.
- *Lãng phí tài nguyên tính toán (Compute Waste):* Đặc thù của thuật toán SDPO là bổ sung thêm nhánh tính toán đắt đỏ (teacher inputs, teacher logits) lên trên quá trình Rollout sinh dữ liệu. Việc thực hiện các tính toán khổng lồ này trên cả những chuỗi lập luận vô nghĩa hoặc bị cắt xén gây ra sự lãng phí nghiêm trọng về thời gian và bộ nhớ GPU.

Do đó, việc cải tiến SDPO để vừa lọc nhiễu, vừa giảm thiểu chi phí tính toán là một yêu cầu cấp thiết.

1.3 Mục tiêu nghiên cứu

Mục tiêu tổng quát: Cải tiến thuật toán SDPO nhằm nâng cao độ tin cậy và khắc phục hiện tượng suy giảm hiệu suất, gia tăng năng lực giải quyết bài toán suy luận toán học của LLMs.

Mục tiêu cụ thể:

- Tích hợp cơ chế Cổng độ tin cậy (Reliability Gate) để đánh giá trọng số tín nhiệm, chỉ lựa chọn các mục tiêu đã được xác minh hoặc có phản hồi an toàn làm dữ liệu chưng cất.
- Cấu hình quy trình huấn luyện phân tán, áp dụng cơ chế thực thi thưa (Sparse

Target Execution) và điều tiết ngân sách lô (Batch Budget) nhằm giảm đáng kể lãng phí tài nguyên tính toán (Compute Waste).

- Huấn luyện các phiên bản thuật toán (Base RL, Vanilla SDPO, Reliability-Gated SDPO) trên nền tảng Verl với mô hình Qwen3, so sánh hiệu suất và thời gian thực thi trên tập dữ liệu DAPO-Math.

1.4 Phương pháp nghiên cứu

Khóa luận sử dụng kết hợp các phương pháp sau:

1. Nghiên cứu tài liệu: Khảo sát, phân tích các nghiên cứu về LLM, RLHF, kỹ thuật Chain-of-Thought và thuật toán SDPO.

2. Nghiên cứu thực nghiệm và đánh giá: Mô hình được thiết lập huấn luyện phân tán trên cụm GPU bằng framework mã nguồn mở Verl/Ray. Quá trình đánh giá được thực hiện thông qua việc so sánh SDPO với Baseline (GRPO) dựa trên các nhóm độ đo (metrics) chuẩn:

- **Độ chính xác (Validation Accuracy):** Đánh giá tỷ lệ giải toán đúng trên tập dữ liệu kiểm thử được trích xuất (held-out subset).
- **Định dạng và Chiều dài (Format & Truncation):** Tỷ lệ đầu ra sai định dạng hoặc bị cắt xén do vượt giới hạn token, kết hợp đánh giá độ dài trung bình của câu trả lời.
- **Hiệu suất hệ thống (Throughput & Execution Time):** Thông lượng đào tạo tổng thể (số token/giây) và thời gian thực thi chi tiết cho các pha sinh dữ liệu (Rollout), tính toán Teacher Logits và cập nhật Actor.
- **Chỉ số Cổng độ tin cậy (SDPO Metrics):** Đo lường tỷ lệ các mẫu được chọn lọc làm mục tiêu SDPO so với lượng ngân sách cho phép, và số phần trăm tính toán (Compute Token Fraction) tiết kiệm được.

1.5 Kết cấu của đề tài và giới hạn đề tài

Giới hạn đề tài:

- *Miền ứng dụng*: Chỉ tập trung vào suy luận toán học nhiều bước.
- *Mô hình nền tảng*: Sử dụng Qwen3 để đảm bảo khả năng tính toán trên phần cứng giới hạn.
- *Bộ dữ liệu*: Thử nghiệm trên tập dữ liệu chuẩn DAPO-Math.

Kết cấu của đề tài: Ngoài phần mở đầu và danh mục, khóa luận gồm 5 chương:

- **Chương 1: Giới thiệu đề tài.** Bối cảnh, lý do, mục tiêu, phương pháp và giới hạn nghiên cứu.
- **Chương 2: Cơ sở lý thuyết.** Kiến thức nền tảng về LLM, Học tăng cường, SDPO và Ver1/vLLM.
- **Chương 3: Phương pháp và giải pháp đề xuất.** Hạn chế của SDPO nguyên bản và kiến trúc thuật toán cải tiến (Reliability Gate).
- **Chương 4: Thực nghiệm và đánh giá.** Môi trường thực nghiệm, cấu hình huấn luyện và phân tích kết quả định lượng.
- **Chương 5: Kết luận và hướng phát triển.** Tóm tắt đóng góp, hạn chế và hướng phát triển tương lai.

Chương 2

CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ TƯƠNG QUAN

2.1 Tổng quan về Học tăng cường (Reinforcement Learning)

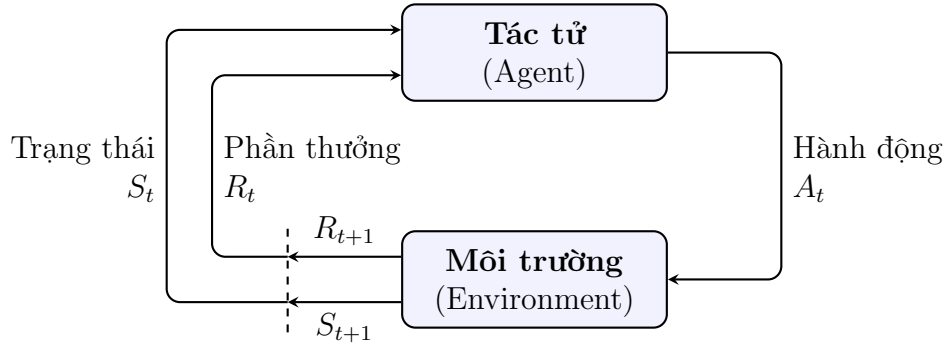
Học tăng cường (Reinforcement Learning - RL) là một nhánh của học máy, trong đó một *tác tử* (*agent*) học cách đưa ra các quyết định thông qua việc tương tác thử-sai với một *môi trường* (*environment*) nhằm tối đa hóa phần thưởng tích lũy (cumulative reward) nhận được.

Về mặt toán học, bài toán học tăng cường được mô hình hóa thành một Quá trình ra quyết định Markov (Markov Decision Process - MDP) được định nghĩa bởi bộ 5 thành phần $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, trong đó:

- $\mathcal{S}$ là tập hợp các *trạng thái* (*states*) của môi trường. Trạng thái tại thời điểm t ký hiệu là S_t .
- $\mathcal{A}$ là tập hợp các *hành động* (*actions*) mà tác tử có thể thực hiện. Hành động tại thời điểm t ký hiệu là A_t .
- $\mathcal{P}(S_{t+1}|S_t, A_t)$ là xác suất chuyển đổi, tức xác suất môi trường chuyển sang trạng thái tiếp theo S_{t+1} khi tác tử thực hiện hành động A_t ở trạng thái hiện tại S_t .
- $\mathcal{R}(S_t, A_t, S_{t+1})$ là hàm *phần thưởng* (*reward*) mang tính phản hồi, trả về giá trị vô hướng R_{t+1} khi tác tử thực hiện hành động A_t .

- $\gamma \in [0, 1]$ là hệ số chiết khấu (discount factor), định lượng tầm quan trọng của các phần thưởng trong tương lai so với phần thưởng hiện tại.

Sự tương tác liên tục và tuần tự này giữa tác tử và môi trường được trực quan hóa chi tiết trong Hình 2.1.



Hình 2.1: Mô hình hóa Quá trình ra quyết định Markov (MDP)

2.1.1 Chính sách và Chính sách tối ưu (Policy & Optimal Policy)

Để tương tác với môi trường và đưa ra quyết định, tác tử tuân theo một *chính sách* (*policy*) π . Có thể hiểu chính sách là một bộ quy tắc, chiến lược cốt lõi của tác tử để quyết định xem nó nên thực hiện hành động nào tiếp theo khi đang ở một trạng thái cụ thể. Trong lý thuyết MDP, chính sách được chia làm hai loại chính:

- **Chính sách xác định (Deterministic Policy):** Ký hiệu là $A_t = \pi(S_t)$. Với mỗi trạng thái S_t , tác tử luôn luôn chọn đúng một hành động duy nhất A_t .
- **Chính sách ngẫu nhiên (Stochastic Policy):** Ký hiệu là $\pi(A_t|S_t) = \mathbb{P}(A_t|S_t)$. Ở đây, chính sách được biểu diễn dưới dạng một phân phối xác suất. Tác tử sẽ lựa chọn hành động A_t thông qua việc lấy mẫu từ phân phối này tại trạng thái S_t . Trong bối cảnh của các Mô hình Ngôn ngữ Lớn (LLMs), các mô hình hoạt động theo chính sách ngẫu nhiên; tại mỗi bước, LLM sinh ra một từ (token) tiếp theo dựa trên phân phối xác suất dự đoán qua toàn bộ không gian từ vựng.

Mục tiêu cốt lõi và tối thượng của bất kỳ bài toán học tăng cường nào là tìm ra một *chính sách tối ưu* (*optimal policy*), ký hiệu là π^* . Một chính sách được coi là tối ưu nếu và chỉ nếu kỳ vọng tổng phần thưởng tích lũy (expected cumulative reward) mà tác tử

thu về khi tuân theo chính sách đó lớn hơn hoặc bằng bất kỳ chính sách nào khác, trên mọi trạng thái có thể có của môi trường. Về mặt toán học, chính sách tối ưu được biểu diễn bằng bài toán tối đa hóa:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^T \gamma^t R_{t+1} \right] \quad (2.1)$$

Trong đó, τ (trajectory) là chuỗi các trạng thái và hành động mà tác tử thực hiện từ đầu đến cuối.

2.1.2 Hàm giá trị (Value Function)

Hàm giá trị $V^{\pi}(s)$ là một thành phần không thể thiếu trong MDP, biểu diễn kỳ vọng tổng phần thưởng tích lũy mà tác tử sẽ nhận được bắt đầu từ trạng thái s và làm theo chính sách π cho đến khi kết thúc quá trình:

$$V^{\pi}(s) = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \middle| S_t = s \right] \quad (2.2)$$

Hàm giá trị giúp tác tử đánh giá xem việc ở tại một trạng thái cụ thể là "tốt" hay "xấu" về mặt dài hạn, từ đó định hướng cho việc cập nhật chính sách π dần tiến tới mức tối ưu π^* .

Về mặt tổng thể, MDP cung cấp một khung toán học chặt chẽ để mô hình hóa quá trình tương tác tuần tự. Trong đó, trạng thái (S_t) biểu diễn tình huống hiện tại của hệ thống, hành động (A_t) đại diện cho quyết định được đưa ra, phần thưởng (R_{t+1}) là tín hiệu định lượng đánh giá chất lượng của quyết định đó, và chính sách (π) đóng vai trò là hàm ánh xạ chiến lược. Quá trình huấn luyện mô hình học tăng cường chính là quá trình tối ưu hóa hàm ánh xạ này để hội tụ về chính sách tối ưu (π^*).

2.2 Mô hình Ngôn ngữ Lớn và Suy luận Toán học

Các Mô hình Ngôn ngữ Lớn (LLMs) được xây dựng dựa trên kiến trúc Transformer, cho phép xử lý và sinh văn bản tự nhiên thông qua việc dự đoán token tiếp theo một cách tự hồi quy (autoregressive). Gọi x là câu hỏi đầu vào và $y = (y_1, y_2, \dots, y_T)$ là chuỗi câu trả

lời, phân phối xác suất của mô hình π_θ được định nghĩa là:

$$\pi_\theta(y|x) = \prod_{t=1}^T \pi_\theta(y_t|x, y_{<t}) \quad (2.3)$$

Trong miền suy luận toán học, quá trình sinh văn bản này được mô hình hóa như một dạng lập luận tự điều chỉnh (*self-Bayesian reasoning*). Ở đó, mỗi bước giải (Chain-of-Thought) phụ thuộc vào cả bài toán gốc và các bước lập luận trước đó, cho phép mô hình liên tục cập nhật niềm tin (belief) của nó về các giả thuyết trung gian. Khác với các tác vụ ngôn ngữ thông thường, toán học đòi hỏi tính chính xác tuyệt đối và khả năng duy trì tính logic qua nhiều bước.

2.3 Học tăng cường: Từ GRPO đến SDPO

2.3.1 Học tăng cường với phần thưởng kiểm chứng (RLVR) và Phản hồi phong phú (RLRF)

Khái niệm Học tăng cường với Phần thưởng Kiểm chứng (Reinforcement Learning from Verifiable Rewards - RLVR) [3] nổi lên như một xu hướng cốt lõi để nâng cao năng lực suy luận toán học của các Mô hình Ngôn ngữ Lớn. Khác với RLHF (dựa vào con người) hay RLAIIF (dựa vào mô hình giám khảo), RLVR tận dụng tín hiệu phần thưởng từ những tác vụ có khả năng tự kiểm chứng (checkable tasks). Cốt lõi của RLVR nằm ở *bộ kiểm chứng (verifier)* — cơ chế tự động trích xuất đáp án (ví dụ: chuỗi nằm trong thẻ `<answer>`) và đánh giá tính tương đương với đáp án chuẩn một cách khách quan.

Tuy nhiên, phần lớn các cách tiếp cận hiện tại của RLVR đều hoạt động dựa trên thiết lập *phần thưởng kết quả vô hướng (Scalar Outcome Reward)*: cho một câu hỏi x , mô hình sinh ra một câu trả lời $y \sim \pi_\theta(\cdot|x)$ và nhận được một phần thưởng vô hướng $r \in \mathbb{R}$, thường là giá trị nhị phân (ví dụ: đúng/sai trong toán học, pass/fail đối với test cases trong lập trình). Việc chỉ dựa vào một con số duy nhất ở cuối cùng này tạo ra một **nút thắt thông tin (information bottleneck)**. Điểm số r sau đó bị phân bổ đồng đều cho mọi token, dẫn đến **bài toán phân bổ tín nhiệm (credit assignment problem)**. Một chuỗi lập luận có bước logic sai nhưng vô tình ra đáp án đúng sẽ khiến mô hình được cộng điểm cho những suy luận sai lầm đó. Ngược lại, một chuỗi đúng 90% nhưng tính sai ở bước cuối cùng lại bị phạt toàn bộ.

Để khắc phục, *phần thưởng quá trình* (*Process Reward*) được đề xuất nhằm đánh giá từng bước trung gian. Dù vậy, việc huấn luyện một Mô hình Phần thưởng Quá trình (PRM) đòi hỏi chi phí gán nhãn khổng lồ. Bên cạnh đó, ở các thuật toán RL truyền thống, việc đánh giá hàm giá trị (value function) cho từng trạng thái trung gian đòi hỏi duy trì một mạng nơ-ron phụ trợ riêng biệt (value network) có kích thước tương đương mô hình chính, làm nhân đôi chi phí bộ nhớ trong quá trình huấn luyện.

Để vượt qua những rào cản này, khái niệm **Học tăng cường với Phản hồi Phong phú (Reinforcement Learning with Rich Feedback - RLRF)** được đề xuất. Thay vì giới hạn ở một tín hiệu vô hướng đơn nhất, RLRF tận dụng các luồng *phản hồi văn bản* (*textual feedback*) được mã hóa dưới dạng token từ môi trường (như thông báo lỗi biên dịch hoặc giải thích chi tiết từ các trình đánh giá). Các phản hồi này không chỉ cung cấp kết quả nhị phân (đúng/sai) mà còn biểu đạt nguyên nhân dẫn đến sự thất bại, từ đó cung cấp ngữ cảnh để mô hình tự điều chỉnh hành vi. Sự khác biệt mang tính bước ngoặt giữa phương pháp phần thưởng vô hướng (RLVR) và phản hồi phong phú (RLRF) được minh họa rõ nét trong Hình 2.2.



Hình 2.2: So sánh thiết lập RLVR và RLRF. Trong RLVR, tác tử học từ một phần thưởng vô hướng r , đóng vai trò như một nút thắt thông tin. Ngược lại, RLRF sử dụng các phản hồi dạng token, cung cấp tín hiệu dồi dào giải thích tại sao một nỗ lực lại thất bại.

Bài toán làm sao để loại bỏ hoàn toàn mạng giá trị tổn kém (Value Network), đồng thời tận dụng phản hồi phong phú để gán tín nhiệm cực kỳ chi tiết (dense credit assignment) chính là tiền đề trực tiếp dẫn đến sự ra đời của các phương pháp tối ưu hóa hiện đại như GRPO và SDPO.

2.3.2 Tối ưu hóa Chính sách Khen thưởng Nhóm (GRPO)

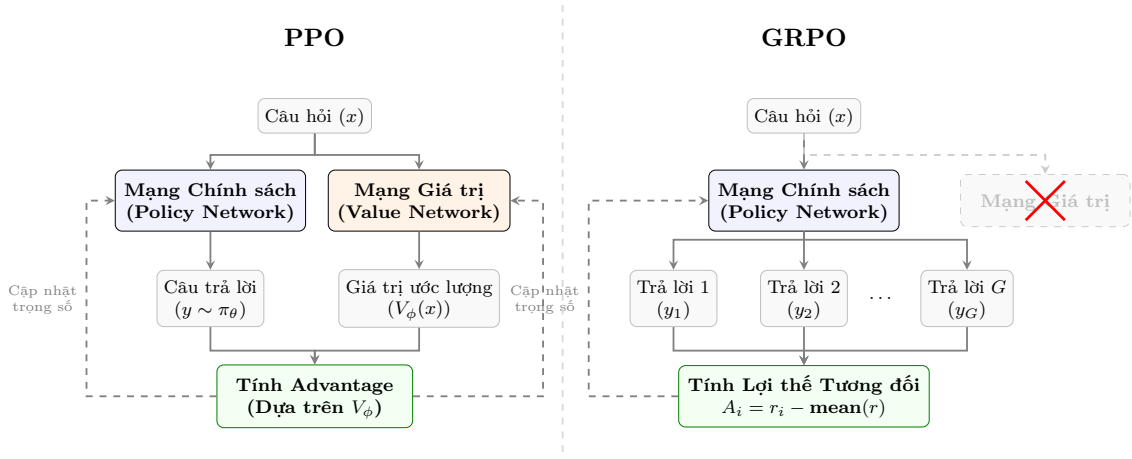
Các thuật toán học tăng cường truyền thống, tiêu biểu là Proximal Policy Optimization (PPO), thường yêu cầu duy trì song song hai mạng nơ-ron: một mạng chính sách (policy network) để đưa ra hành động và một mạng giá trị (value network) để ước lượng hàm giá trị. Do mạng giá trị thường có số lượng tham số tương đương với mạng chính sách, thiết lập này làm tăng gấp đôi chi phí bộ nhớ GPU trong quá trình huấn luyện.

Group Relative Policy Optimization (GRPO) là một thuật toán tối ưu hóa giải quyết triệt để hạn chế trên bằng cách loại bỏ hoàn toàn mạng giá trị. Thay vì dựa vào một mô hình phụ trợ để ước lượng giá trị tuyệt đối làm đường cơ sở (baseline), GRPO thực hiện lấy mẫu một nhóm (group) gồm G câu trả lời cho cùng một câu hỏi x , chấm điểm chúng bằng phần thưởng r_i , và trực tiếp sử dụng giá trị trung bình của nhóm làm baseline. Từ đó, mô hình tính toán *lợi thế tương đối* (relative advantage) của từng câu trả lời trong phạm vi nhóm:

$$A_{i,t}^{\text{GRPO}} := r_i - \text{mean}\{r_j\}_{j=1}^G \quad (2.4)$$

Trong đó, r_i là phần thưởng vô hướng của câu trả lời thứ i , và việc lấy trung bình được thực hiện trên toàn bộ G mẫu trong nhóm để làm đường cơ sở (baseline) so sánh.

Hình 2.3 thể hiện rõ sự khác biệt về mặt kiến trúc: GRPO loại bỏ hoàn toàn mạng giá trị công kênh của PPO, qua đó tiết kiệm đáng kể bộ nhớ và tăng tốc độ huấn luyện.



Hình 2.3: Minh họa sự khác biệt kiến trúc giữa PPO (đòi hỏi Value Network) và GRPO (loại bỏ hoàn toàn Value Network thông qua lấy mẫu nhóm).

Lợi thế này sau đó được tích hợp trực tiếp vào hàm mục tiêu tối ưu hóa chính sách (surrogate objective), kết hợp với một hình phạt Kullback-Leibler (KL) nhằm đảm bảo

mô hình không cập nhật quá xa so với chính sách tham chiếu (reference policy) ban đầu:

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{x, y \sim \pi_\theta} \left[\sum_t \left(\min \left(\frac{\pi_\theta(y_t | x, y_{<t})}{\pi_{\text{old}}(y_t | x, y_{<t})} A^{\text{GRPO}}, \text{clip}(\dots) A^{\text{GRPO}} \right) - \beta_{\text{KL}} \right) \right] \quad (2.5)$$

Trong đó:

- θ là các tham số của mô hình đang được tối ưu hóa.
- π_θ là chính sách hiện tại, và π_{old} là chính sách ở bước cập nhật trước.
- Tỷ lệ $\frac{\pi_\theta}{\pi_{\text{old}}}$ xác định mức độ thay đổi của chính sách.
- Hàm clip giúp giới hạn sự thay đổi này trong một phạm vi an toàn, tránh cập nhật quá tay.
- β là hệ số phạt KL divergence.

Mặc dù tối ưu về mặt tính toán, GRPO mắc phải **nút thắt phân bổ tín nhiệm (credit-assignment bottleneck)**. Do lợi thế A^{GRPO} là một hằng số đối với mọi token t trong chuỗi y_i , mô hình sẽ gán chung một mức điểm thưởng/phạt cho toàn bộ các token. Nếu mô hình làm sai ở bước thứ 3 trong một bài toán cần 10 bước, nó vẫn sẽ nhận điểm 0 và không thể biết được rằng các bước sau đó có thể là những lập luận logic hợp lý dựa trên một tiền đề sai.

2.3.3 Tối ưu hóa Chính sách Tự chưng cất (SDPO)

Để giải quyết bài toán phân bổ tín nhiệm của GRPO, Self-Distillation Policy Optimization (SDPO) [1] sử dụng chính phản hồi f để xây dựng một "teacher" tự thân (self-teacher). Theo thiết lập của SDPO, tín hiệu phản hồi f thường kết hợp hai loại thông tin cốt lõi: (1) phản hồi trực tiếp từ môi trường đánh giá (ví dụ: kết quả báo lỗi logic hoặc sai đáp án cuối cùng) và (2) một lời giải mẫu (sample solution) trong trường hợp câu hỏi x đã được giải thành công bởi một nỗ lực khác trong cùng nhóm (rollout group).

Thay vì huấn luyện thêm một mạng giá trị cấp độ token phức tạp, SDPO định nghĩa chính sách của teacher là $\pi_{\text{ref}}(\cdot | x, f)$, tức là mô hình được cung cấp thêm thông tin phản hồi trong ngữ cảnh (in-context) để đánh giá lại câu trả lời. Cơ chế này xuất phát từ một quan sát quan trọng: *chúng ta có thể sử dụng cùng một mạng nơ-ron cho hai vai trò. Ở*

lượt giải đầu tiên, nó đóng vai trò "student" $\pi_\theta(\cdot|x)$; sau khi nhận phản hồi, nó chuyển vai thành "teacher" với khả năng đánh giá sắc bén hơn nhờ góc nhìn hồi tố (hindsight).

Hàm mất mát của SDPO tối ưu hóa khoảng cách Kullback-Leibler (KL divergence) giữa "student" (không có phản hồi) và "teacher" (có phản hồi):

$$\mathcal{L}_{\text{SDPO}}(\theta) := \sum_t \text{KL}(\pi_\theta(\cdot|x, y_{<t}) \parallel \text{stopgrad}(\pi_{\text{ref}}(\cdot|x, f, y_{<t}))) \quad (2.6)$$

Trong đó:

- $\mathcal{L}_{\text{SDPO}}(\theta)$ là hàm mất mát chưng cất tự thân.
- t là chỉ số (vị trí) của token hiện tại đang được sinh ra.
- $y_{<t}$ đại diện cho toàn bộ chuỗi các token đã được sinh ra trước bước t .
- $\sum_t$ biểu thị việc lấy tổng hàm mất mát trên tất cả các bước sinh token của câu trả lời.
- $\pi_\theta(\cdot|x, y_{<t})$ là phân phối xác suất sinh token thứ t của Student (chỉ nhìn thấy câu hỏi x và các token trước đó $y_{<t}$).
- $\pi_{\text{ref}}(\cdot|x, f, y_{<t})$ là phân phối xác suất sinh token thứ t của Teacher (được thêm phản hồi f vào ngữ cảnh).
- stopgrad dùng để đóng băng đạo hàm của Teacher, chỉ cập nhật trọng số cho Student.

Bằng cách lấy đạo hàm của hàm mất mát này và ánh xạ nó vào khuôn khổ PPO, quá trình này hình thành một lợi thế tín nhiệm ở cấp độ logit (dense logit-level advantage):

$$A_{i,t}^{\text{SDPO}}(\hat{y}_{i,t}) = \log \frac{\pi_{\text{ref}}(\hat{y}_{i,t}|x, f_i, y_{i,<t})}{\pi_\theta(\hat{y}_{i,t}|x, y_{i,<t})} \quad (2.7)$$

Nhờ tín hiệu học tập dày đặc này, SDPO giúp mô hình biết chính xác cần sửa lỗi ở token nào (token nào có xác suất thấp hơn so với teacher). Hơn nữa, để cân bằng giữa sự chưng cất (distillation) và phần thưởng từ môi trường truyền thống, lợi thế của SDPO được lai (hybrid) với GRPO:

$$A_{i,t}^{\text{Hybrid}}(\hat{y}_{i,t}) := \lambda A_{i,t}^{\text{GRPO}}(\hat{y}_{i,t}) + (1 - \lambda) A_{i,t}^{\text{SDPO}}(\hat{y}_{i,t}) \quad (\lambda \in [0, 1]) \quad (2.8)$$

Trong đó λ là siêu tham số dùng để nội suy (nếu $\lambda = 1$ thì quay về GRPO thuần túy, nếu $\lambda = 0$ thì là SDPO thuần túy).

Toàn bộ quá trình huấn luyện SDPO được tóm tắt trong Mã giả dưới đây:

Algorithm 1 Tối ưu hóa Chính sách Tự chưng cất (SDPO)

Require: Mô hình ngôn ngữ π_θ ; tập dữ liệu chứa các câu hỏi x ; số lượng rollouts G cho mỗi câu hỏi; môi trường để lấy phản hồi cho các câu trả lời.

- 1: **repeat**
 - 2: Lấy mẫu câu hỏi x từ tập dữ liệu.
 - 3: Lấy mẫu các câu trả lời: $\{y_i\}_{i=1}^G \sim \pi_\theta(\cdot|x)$.
 - 4: Đánh giá các câu trả lời để nhận phản hồi f_i .
 - 5: ▷ *Tự chưng cất (Self-distillation)*:
 - 6: Tính log-probs của teacher $\log \pi_\theta(y_{i,t}|x, f_i, y_{i,<t})$.
 - 7: Cập nhật θ bằng gradient descent trên $\mathcal{L}_{\text{SDPO}}(\theta)$.
 - 8: **until** hội tụ (converged)
-

Thuật toán 1 trình bày một vòng lặp hoàn chỉnh của SDPO. So với các phương pháp chưng cất tri thức truyền thống thường đòi hỏi một mô hình teacher (teacher model) với số lượng tham số vượt trội, điểm đột phá của SDPO nằm ở việc liên tục cập nhật chính sách tham chiếu từ chính mô hình hiện tại thông qua việc đóng băng đạo hàm (**stopgrad**). Cơ chế này giúp mô hình tự điều chỉnh sai lệch dựa trên sự phân kỳ giữa hai phân phối xác suất: một phân phối thiếu hụt thông tin ngữ cảnh (đóng vai trò student) và một phân phối được cung cấp đầy đủ thông tin hướng dẫn từ phản hồi văn bản (đóng vai trò teacher).

Để ánh xạ quá trình chưng cất này vào khuôn khổ tối ưu hóa của Học tăng cường (RL), nhóm tác giả đã chứng minh toán học sự tương đương giữa đạo hàm của hàm mất mát Kullback-Leibler và một hàm lợi thế (advantage) cấp độ token:

Mệnh đề 2.1 (Proposition 2.1). Đạo hàm của $\mathcal{L}_{\text{SDPO}}$ được biểu diễn dưới dạng:

$$\nabla \mathcal{L}_{\text{SDPO}}(\theta) = \mathbb{E}_{y \sim \pi_\theta(\cdot|x)} \left[\sum_{t=1}^{|y|} \mathbb{E}_{\hat{y}_t \sim \pi_\theta(\cdot|x, y_{<t})} \left[\log \frac{\pi_\theta(\hat{y}_t|x, y_{<t})}{\pi_\theta(\hat{y}_t|x, f, y_{<t})} \cdot \nabla_\theta \log \pi_\theta(\hat{y}_t|x, y_{<t}) \right] \right]. \quad (2.9)$$

Thông qua thủ thuật hàm điểm số (score function trick), Mệnh đề 2.1 minh chứng rằng SDPO thực chất là một phương pháp Gradient Chính sách (Policy Gradient). Trong đó, thay vì sử dụng phần thưởng vô hướng truyền thống như GRPO, SDPO sử dụng *tỷ lệ log-xác suất (log-probability ratio)* giữa mô hình teacher và mô hình student làm hàm lợi thế (advantage function). Cụ thể, các token $\hat{y}_t$ được mô hình teacher đánh giá với xác suất cao hơn (nhờ có thêm thông tin phản hồi f) sẽ nhận được giá trị lợi thế dương, từ

đó gia tăng độ tin cậy để mô hình tiếp tục sinh ra token đó trong các vòng lặp sau. Ngược lại, lợi thế âm sẽ được gán cho các token dẫn đến bước suy luận sai lệch.

Một đặc tính thực nghiệm quan trọng của hàm lợi thế này là sự *kích hoạt thưa thớt* (*sparse activation*). Tại các bước lập luận hoàn toàn hợp lý và chính xác, tỷ lệ log-xác suất giữa teacher và student gần như bằng 0 (dẫn đến gradient không biến thiên). Lợi thế chỉ ghi nhận giá trị đột biến tại chính xác vị trí token xảy ra lỗi logic (error tokens) hoặc nơi mô hình thực sự cần điều chỉnh. Khả năng tự nhận diện sai lầm thông qua hồi tố (retrospection) một cách thưa thớt (sparse) mà không cần nhãn giám sát cấp độ token (token-level labels) chính là ưu điểm cốt lõi giúp SDPO giải quyết triệt để nút thắt phân bổ tín nhiệm.

Tuy nhiên, hiệu quả của cơ chế tự chứng cất này phụ thuộc đáng kể vào phương pháp mô hình tiếp nhận thông tin phản hồi. Để mô hình teacher tự thân (self-teacher) có thể cung cấp các tín hiệu hướng dẫn chuẩn xác thông qua học tập trong ngữ cảnh (in-context learning), một khuôn mẫu (template) đặc biệt đã được thiết kế như trình bày chi tiết trong Bảng 2.1.

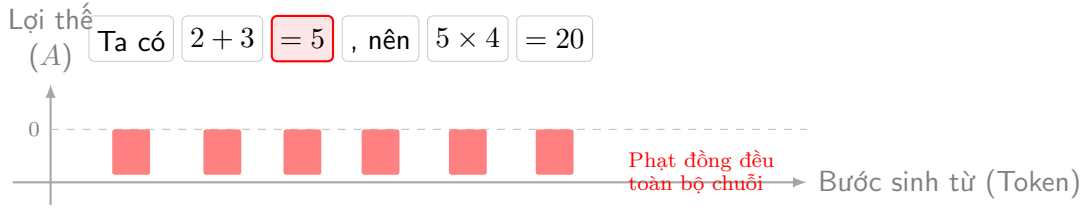
Bảng 2.1: Khuôn mẫu (Template) dành cho "teacher". **prompt** được thay bằng câu hỏi. **successful_previous_rollout** là lời giải mẫu chính xác được sinh ra trong cùng nhóm (sẽ bị bỏ qua đoạn này nếu không có). **environment_output** là phản hồi lỗi từ môi trường đối với nỗ lực giải ban đầu (bỏ qua nếu đã có lời giải mẫu). Nếu nỗ lực ban đầu là đúng, nó sẽ được chuyển thành lời giải mẫu. Cuối cùng, **original_response** được thay bằng câu trả lời gốc của mô hình để tính toán lại xác suất (log-probs) dưới lăng kính của teacher.

User:	prompt Lời giải chính xác: successful_previous_rollout Sau đây là phản hồi từ nỗ lực giải quyết không thành công trước đó của bạn: environment_output Hãy giải quyết chính xác câu hỏi ban đầu.
Assistant:	original_response

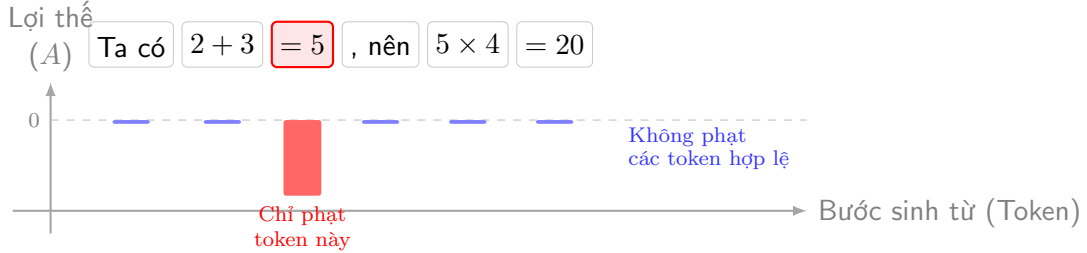
Bằng cách lồng ghép các yếu tố trên vào ngữ cảnh, mô hình được định hướng thực hiện quá trình *đánh giá hồi tố* (*retrospection*) đối với nỗ lực gốc của chính mình. Quá trình này tạo ra một tín hiệu học tập ở cấp độ token (dense token-level signal), góp phần trực tiếp tháo gỡ nút thắt phân bổ tín nhiệm. Để hiểu rõ hơn về sự ưu việt của cơ chế này, Hình 2.4 minh họa trực quan cách SDPO gán tín nhiệm khắt khe vào đúng token gây ra lỗi logic, trái ngược hoàn toàn với việc phạt đồng đều toàn chuỗi của GRPO.

Câu hỏi: Tính $2 + 3 \times 4$

1. GRPO (Phân bổ tín nhiệm toàn chuỗi - Sequence-level):



2. SDPO (Gán tín nhiệm từng token - Dense logit-level):



Hình 2.4: Minh họa sự khác biệt trong việc gán tín nhiệm giữa GRPO và SDPO đối với một bài toán suy luận Toán học. GRPO phạt toàn bộ chuỗi khi kết quả cuối cùng sai, trong khi cơ chế tự chưng cất của SDPO giúp phát hiện chính xác token gây ra lỗi logic (kích hoạt thừa thớt) và bỏ qua các phần suy luận hợp lệ theo sau.

2.4 Biểu đạt nhận thức (Epistemic Verbalization) và Rủi ro triệt tiêu

Để hiểu được vì sao các phương pháp Tự chưng cất (Self-Distillation) như SDPO đôi khi lại làm **suy giảm** năng lực suy luận toán học, nghiên cứu gần đây của [2] đã chỉ ra nguồn gốc của vấn đề nằm ở khái niệm "Biểu đạt nhận thức" và sự phong phú của lượng tin tương hỗ.

2.4.1 Khái niệm Biểu đạt nhận thức trong suy luận Toán học

Trong các Mô hình Ngôn ngữ Lớn (LLMs), suy luận toán học có thể được xem như một dạng lập luận tự điều chỉnh (self-Bayesian reasoning), nơi mô hình liên tục sinh ra từng bước giải và cập nhật niềm tin (belief) về các giả thuyết trung gian. Trong quá trình này, các LLMs mạnh thường chèn thêm các từ ngữ thể hiện sự không chắc chắn như "wait", "hmm", "perhaps", "maybe".

Việc sử dụng các từ khóa này được gọi là **Biểu đạt nhận thức (Epistemic Verbalization)**. Đây không phải là sự dài dòng vô nghĩa, mà là cách mô hình ngoại hóa sự không chắc chắn (uncertainty externalization). Nhờ có chúng, mô hình duy trì được các

hướng giải thay thế, tránh việc "cam kết sớm" (prematurely commit) vào một lập luận sai lầm với rất ít cơ hội để tự sửa sai (recovery).

2.4.2 Lượng tin tương hỗ và Sự thay đổi hành vi suy luận

Trong thiết lập Tự chứng cất (như SDPO), mô hình "teacher" được cung cấp thêm một ngữ cảnh c (chẳng hạn như một đáp án đúng, phản hồi từ môi trường). Mức độ phong phú của ngữ cảnh này được đo bằng **Lượng tin tương hỗ có điều kiện (Conditional Mutual Information)**:

$$I(y; c|x) = H(y|x) - H(y|x, c) \quad (2.10)$$

Lượng tin $I(y; c|x)$ này đo lường mức độ giảm thiểu sự không chắc chắn về câu trả lời y khi biết thêm ngữ cảnh c . Bài báo [2] phân tích 4 mức độ cung cấp thông tin, từ thấp đến cao:

1. Sinh tự do (Unguided): $c = \emptyset \implies I(y; \emptyset|x) = 0$.
2. Hướng dẫn bằng đáp án đã loại bỏ bước suy nghĩ ($s_{\backslash \text{think}}$).
3. Hướng dẫn bằng lời giải tự sinh ($\tilde{y}$).
4. Hướng dẫn bằng đáp án hoàn chỉnh (s). Mức này có lượng tin cao nhất.

Điều này dẫn đến bất đẳng thức: $I(y; \emptyset|x) = 0 < I(y; s_{\backslash \text{think}}|x) \leq I(y; \tilde{y}|x) \leq I(y; s|x)$.

Nguyên lý cốt lõi rút ra: Khi ngữ cảnh c càng cung cấp nhiều thông tin trực tiếp hữu ích, mô hình càng trở nên tự tin. Kết quả là nó sinh ra câu trả lời ngắn gọn hơn và **đánh mất đi các biểu đạt nhận thức (epistemic verbalization giảm mạnh)**.

2.4.3 Hiện tượng Triệt tiêu nhận thức (Epistemic Suppression)

Khi huấn luyện mô hình "student" (vốn không có ngữ cảnh phong phú c) phải học theo phong cách trả lời ngắn gọn, tự tin của "teacher" (vốn đã biết trước đáp án $c = s$), mô hình student bị ép phải loại bỏ các biểu đạt nhận thức. Quá trình này được gọi là hiện tượng **Triệt tiêu nhận thức**.

Hậu quả của việc này là mô hình student bị học một phong cách suy luận quá tự tin (overconfident) và đánh mất tín hiệu dò dẫm tự nhiên (như "wait", "hmm"). [2] chứng minh rằng dù student được huấn luyện bằng các chuỗi đáp án hoàn toàn đúng, việc thiếu

đi biểu đạt nhận thức khiến hiệu suất giải toán giảm sút nghiêm trọng trên các bài toán chưa từng gặp (Out-of-Distribution - OOD) (giảm tới 40% trên tập AIME24).

Tác động của hiện tượng này phụ thuộc mạnh vào **Độ phủ của bài toán (Task Coverage)**:

- **Ở các miền có độ phủ hẹp (như Hóa học, Lập trình ứng dụng):** Việc cắt bỏ các bước thăm dò giúp mô hình đi thẳng vào vấn đề, làm tăng tính hiệu quả (Efficiency) và điểm số.
- **Ở miền Toán học:** Do tính đa dạng khổng lồ của các dạng toán (độ phủ lớn), việc triệt tiêu khả năng tự vấn khiến mô hình mất đi tính thích ứng (adapt) với các bài toán OOD, làm giảm hiệu suất tổng thể.

Sự đánh đổi (trade-off) giữa tính sức tích và khả năng suy luận linh hoạt trong các môi trường có độ phủ rộng chính là rào cản lớn nhất của SDPO. Phát hiện này là tiền đề cốt lõi để đề tài đề xuất *Cổng độ tin cậy (Reliability Gate)*. Cổng này đóng vai trò như một bộ lọc (filter), điều tiết quá trình chứng cất dựa trên sự tin cậy của người thầy, nhằm giới hạn sự triệt tiêu nhận thức thái quá và đảm bảo mô hình vẫn duy trì được năng lực tự khám phá khi giải Toán.

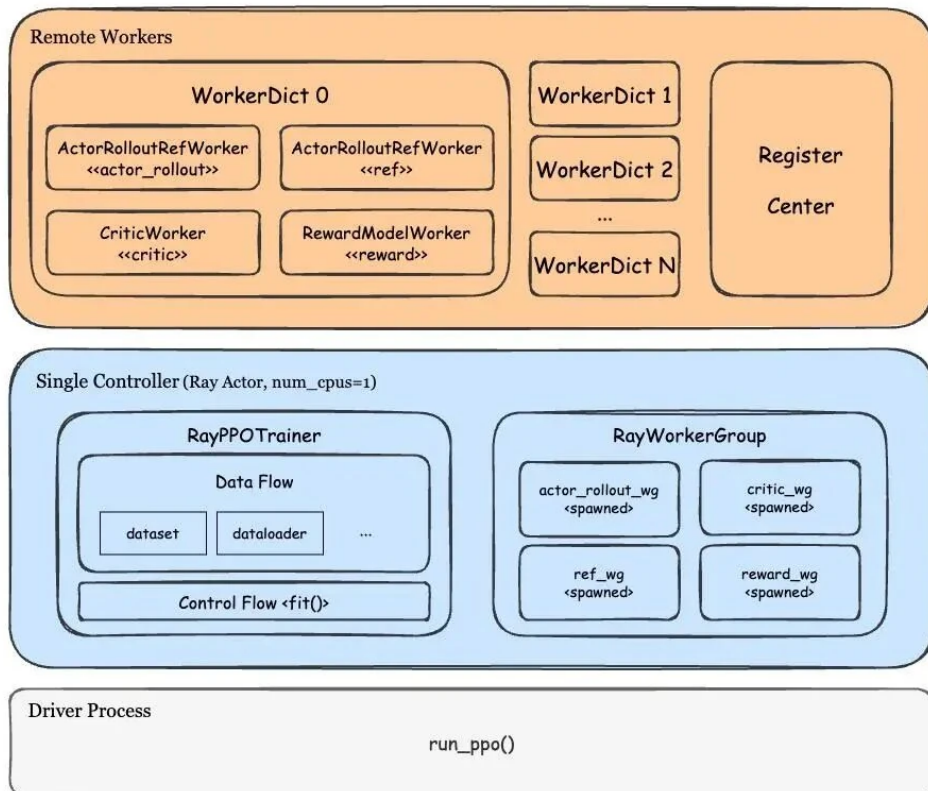
2.5 Các công nghệ và nền tảng hỗ trợ

Khóa luận sử dụng **Verl** - một framework mã nguồn mở linh hoạt dành cho việc huấn luyện RL phân tán trên LLMs. Điểm khác biệt lớn nhất của Verl so với các framework như OpenRLHF là việc sử dụng kiến trúc **Bộ điều khiển trung tâm duy nhất (Single Controller)**. Thay vì phân tán logic điều khiển cho các module Actor (dễ gây nghẽn cổ chai), Verl sử dụng một controller độc lập trên CPU đóng vai trò như "bộ não" quản lý toàn bộ luồng dữ liệu, luồng điều khiển và khởi tạo tài nguyên thông qua nền tảng Ray (kiến trúc tổng thể được mô tả trong Hình 2.5)¹.

Cốt lõi của kiến trúc này nằm ở cấu trúc **WorkerDict** - một lớp cơ sở hoạt động như một Worker đa năng. Nhờ khả năng liên kết (bind) phương thức của tất cả các module thành phần (Actor, Critic, Reward, v.v.), WorkerDict có thể chuyển đổi vai trò một cách linh hoạt (*Hybrid Engine*). Đặc biệt, module Actor (huấn luyện) và Rollout (suy luận)

¹Nguồn tham khảo kiến trúc: <https://langcopilot.com/posts/2025-11-06-openrlhf-vs-verl-ray-framework-deep>

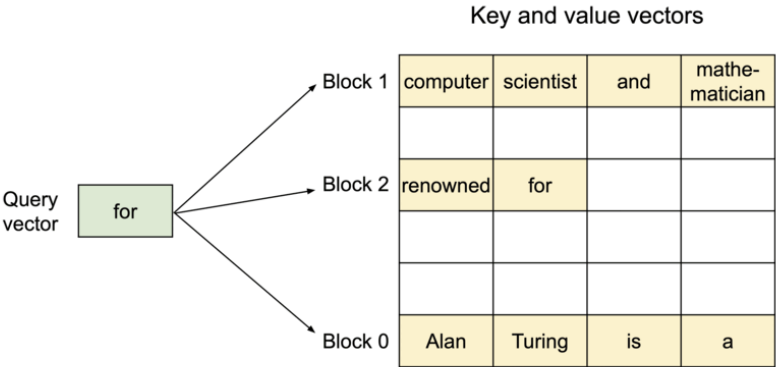
trong Verl được thiết kế để chia sẻ chung trọng số mô hình ngay trên bộ nhớ (shared memory). Cơ chế này loại bỏ hoàn toàn nhu cầu đồng bộ trọng số qua mạng giao tiếp (như CUDA IPC trong OpenRLHF), từ đó tối ưu hóa vượt trội hiệu năng và bộ nhớ cho quá trình huấn luyện RL quy mô lớn.



Hình 2.5: Kiến trúc phân tán của Verl dựa trên nền tảng Ray.

Đặc biệt, để tối ưu hóa quá trình Rollout (giai đoạn mô hình tự sinh lời giải để lấy mẫu học tập), hệ thống tích hợp **vLLM** - một động cơ suy luận (inference engine) hiệu năng cao. Nhờ công nghệ PagedAttention cấp phát bộ nhớ động cho KV Cache, vLLM cho phép thực hiện Offline Batched Inference (sinh văn bản theo lô lớn) với tốc độ vượt trội, giải quyết triệt để nút thắt về thời gian sinh dữ liệu khổng lồ của các thuật toán RL (cơ chế này được minh họa ở Hình 2.6)².

²Nguồn tham khảo kiến trúc: <https://blog.vllm.ai/2023/06/20/vllm.html>



Hình 2.6: Cơ chế cấp phát bộ nhớ động PagedAttention của vLLM giúp tăng tốc độ sinh dữ liệu.

Chương 3

PHƯƠNG PHÁP ĐỀ XUẤT VÀ THIẾT KẾ

Chương này trình bày chi tiết về cải tiến cốt lõi của khóa luận đối với thuật toán SDPO: **Self-Distillation Policy Optimization kết hợp Cổng độ tin cậy (Reliability-Gated SDPO)**. Đây là phương pháp chủ đạo được áp dụng để giải quyết các thách thức đặc thù trong bài toán suy luận toán học.

3.1 Giới hạn của SDPO truyền thống trong lĩnh vực Toán học

Thuật toán SDPO nguyên bản (Vanilla SDPO) hoạt động dựa trên một giả định cốt lõi: mọi mục tiêu chưng cất (distillation targets) được tạo ra từ phản hồi (feedback-reprompted) đều có ích cho quá trình học tập. Giả định này có thể đúng trong một số tác vụ đơn giản, nhưng lại bộc lộ điểm yếu nghiêm trọng trong lĩnh vực toán học.

Trong giải toán, một câu trả lời sai không đơn thuần chỉ là kết quả sai, mà nó có thể bao gồm một chuỗi suy luận logic rất dài, trôi chảy nhưng chứa đựng các bước lập luận sai lầm (misleading reasoning). Việc SDPO truyền thống mù quáng ép mô hình học trò "bắt chước" (distill) những chuỗi suy luận kém chất lượng này sẽ vô tình củng cố tư duy sai lệch, dẫn đến hiện tượng triệt tiêu nhận thức và làm suy giảm năng lực tự khám phá của mô hình.

3.2 Đề xuất SDPO kết hợp Cổng độ tin cậy (Reliability-Gated SDPO)

Để khắc phục điểm yếu trên, phương pháp Reliability-Gated SDPO vẫn giữ nguyên cơ chế chứng cất tự thân của SDPO, nhưng can thiệp sâu vào khâu lựa chọn mục tiêu bằng một "Cổng độ tin cậy" (Reliability Gate) nhạy bén với phản hồi.

3.2.1 Hàm mục tiêu (Objective Function)

Giả sử x_i là bài toán, y_i là chuỗi câu trả lời của mô hình, $y_{i,<t}$ là các token đã sinh ra trước bước t . f_i là tín hiệu phản hồi (kết hợp đúng/sai và lời giải mẫu nếu có). π_θ là chính sách hiện tại của mô hình.

Hàm mục tiêu của RL cơ bản (Base RL) chỉ tối ưu hóa dựa trên lợi thế:

$$L_{base}(\theta) = L_{RL}(\pi_\theta) \quad (3.1)$$

Thuật toán Vanilla SDPO nguyên bản bổ sung thêm một hàm mất mát chứng cất tự thân (tính bằng phân kỳ Kullback-Leibler - KL) áp dụng trên *tất cả* các mục tiêu có sẵn:

$$L_{vanilla}(\theta) = L_{RL}(\pi_\theta) + \lambda \times \mathbb{E}_{i,t} [m_i \cdot \text{KL}(\pi_\theta(\cdot|x_i, y_{i,<t}) || \text{stopgrad}(\pi_\theta(\cdot|x_i, f_i, y_{i,<t})))] \quad (3.2)$$

Phương pháp **Reliability-Gated SDPO** đề xuất thay đổi hàm mục tiêu bằng cách đưa vào hai biến số mới là **trọng số tín nhiệm** (w_i) và **cổng nhị phân** (g_i):

$$L_{gate}(\theta) = L_{RL}(\pi_\theta) + \lambda \times \mathbb{E}_{i,t} [m_i \cdot g_i \cdot w_i \cdot \text{KL}(\pi_\theta(\cdot|x_i, y_{i,<t}) || \text{stopgrad}(\pi_\theta(\cdot|x_i, f_i, y_{i,<t})))] \quad (3.3)$$

Giải thích chi tiết các thành phần trong công thức:

- $L_{gate}(\theta)$: Tổng hàm mục tiêu tối ưu hóa của hệ thống dựa trên bộ tham số θ .
- $L_{RL}(\pi_\theta)$: Hàm mất mát của Học tăng cường (được tính dựa trên lợi thế GRPO/PPO để cực đại hóa phần thưởng).
- λ : Hệ số siêu tham số (hyperparameter) cân bằng mức độ ảnh hưởng của nhánh tự chứng cất so với nhánh RL gốc.

- $\mathbb{E}_{i,t}$: Kỳ vọng (giá trị trung bình) được tính toán trên toàn bộ các mẫu i trong lô dữ liệu (batch) và từng token t trong chuỗi.
- m_i : Mặt nạ mục tiêu gốc của SDPO ($m_i = 1$ nếu mục tiêu hợp lệ để chứng cất, $m_i = 0$ nếu bị cắt xén/vượt quá token).
- $g_i \in \{0, 1\}$: Cổng kiểm duyệt độ tin cậy. Chỉ mở ($g_i = 1$) khi hệ thống tin tưởng độ an toàn của phản hồi f_i .
- $w_i \in [0, 1]$: Trọng số tín nhiệm (điều tiết lực chứng cất mạnh hay yếu tùy thuộc vào chất lượng của lời giải).
- t và $y_{i,<t}$: t là chỉ số của token hiện tại đang được sinh ra, còn $y_{i,<t}$ là toàn bộ các token đã sinh ra trước đó của câu trả lời y_i . Do mô hình ngôn ngữ sinh văn bản theo kiểu tự hồi quy (autoregressive), xác suất của token hiện tại luôn phụ thuộc vào câu hỏi và các token trước nó.
- $\pi_\theta(\cdot|x_i, y_{i,<t})$: Xác suất sinh token thứ t của mô hình *Student* (không được xem phản hồi).
- $\pi_\theta(\cdot|x_i, f_i, y_{i,<t})$: Xác suất sinh token thứ t của mô hình *Teacher* (chính là *Student* nhưng được cung cấp thêm thông tin f_i để có "góc nhìn hồi tố").
- `stopgrad(...)`: Lệnh dừng lan truyền ngược (gradient), nhằm đóng băng các tham số của *Teacher*, ép *Student* phải học theo *Teacher* chứ không phải ngược lại.

Trong đó, mô hình vẫn học hỏi từ phần thưởng RL, nhưng hàm mất mát tự chứng cất chỉ được kích hoạt ($g_i = 1$) và nhận lực cập nhật tương ứng (w_i) đối với những mục tiêu thực sự đáng tin cậy.

3.2.2 Cơ chế gán trọng số tín nhiệm (Reliability Weights)

Hệ thống gán trọng số w_i cho từng mục tiêu SDPO dựa trên phản hồi của hệ thống chấm điểm (math verifier), cụ thể như Bảng 3.1:

Trường hợp (Case)	Trọng số (w_i)	Ý nghĩa và Lý do
Lời giải đúng đã xác minh	1.0	Mục tiêu mạnh nhất; đã vượt qua hệ thống kiểm chứng phần thưởng.
Phản hồi tính đúng đắn an toàn	0.4	Tín hiệu sửa lỗi hữu ích, nhưng không hoàn hảo như lời giải đã xác minh.
Phản hồi về định dạng	0.2	Giúp ích cho định dạng, nhưng yếu về mặt chất lượng suy luận logic.
Đầu ra bị cắt xén (Truncated)	0.0	Mục tiêu không an toàn; tuyệt đối không nên để mô hình bất chước.
Không có lời giải & phản hồi	0.0	Không có mục tiêu SDPO nào hữu ích.

Bảng 3.1: Bảng phân bổ Trọng số tín nhiệm cho các mục tiêu SDPO.

3.2.3 Cổng lọc và Ngân sách tính toán (Gate & Batch Budget)

Giá trị của cổng nhị phân g_i được quyết định dựa trên một ngưỡng τ và ngân sách của một lô (batch budget) được định nghĩa qua tỷ lệ chọn tối đa (max selected fraction):

$$g_i = \mathbf{1}[w_i \geq \tau] \wedge \text{selected_by_batch_budget}(i) \quad (3.4)$$

Với cấu hình mặc định, khóa luận thiết lập $\tau = 0.4$ và **Tỷ lệ chọn tối đa** = 0.5. Điều này có nghĩa là Cổng lọc sẽ tự động từ chối hoàn toàn các mục tiêu thuần túy sửa lỗi định dạng ($w_i = 0.2$) và các mục tiêu bị cắt xén ($w_i = 0.0$). Ngay cả khi có rất nhiều mục tiêu vượt qua ngưỡng τ , hệ thống cũng chỉ giữ lại tối đa 50% số lượng mẫu có độ tin cậy cao nhất trong một lô để tiết kiệm tính toán.

3.3 Các khía cạnh cải tiến đột phá

Thuật toán Reliability-Gated SDPO mang lại hai sự cải tiến vượt bậc so với SDPO truyền thống:

3.3.1 Nâng cao chất lượng mục tiêu (Higher-quality SDPO targets)

Hàm mất mát SDPO giờ đây được tập trung hoàn toàn vào các mục tiêu đã được xác minh hoặc được dẫn xuất từ các phản hồi an toàn. Việc chủ động loại bỏ các mục tiêu có độ tin cậy thấp (những lời giải sai, nhiễu) giúp mô hình tránh khỏi việc mô phỏng các suy luận sai lệch, từ đó nâng cao chất lượng học tập và độ chính xác của mô hình khi giải

quyết các bài toán học búa.

3.3.2 Giảm thiểu chi phí tính toán (Lower SDPO compute)

Đặc thù của thuật toán SDPO là tiêu tốn lượng tài nguyên khổng lồ do phải cộng dồn thêm các đầu vào của Teacher, tính toán logits của Teacher và cập nhật Actor. Bằng việc kết hợp thực thi mục tiêu thưa (sparse target execution), hệ thống bỏ qua toàn bộ việc tính toán hàm mất mát SDPO ở nhánh đắt đỏ cho các mẫu có độ tin cậy thấp. Điều này không chỉ bảo vệ mô hình khỏi dữ liệu rác mà còn giảm đáng kể thời gian và vRAM cần thiết cho quá trình huấn luyện.

Chương 4

THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1 Môi trường và dữ liệu thực nghiệm

4.2 Thiết lập tham số huấn luyện

4.3 Kết quả thực nghiệm

4.4 Phân tích và đánh giá kết quả

Chương 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

5.2 Những đóng góp chính

5.3 Hướng phát triển tiếp theo

Tài liệu tham khảo

- [1] Hübotter, J., Lübeck, F., Behric, L., Baumann, A., Bagatella, M., Marta, D., Hakimi, I., Shenfeld, I., Buening, T. K., Guestrin, C., & Krause, A. *Reinforcement Learning via Self-Distillation*. 2026.
- [2] Kim, J., Luo, X., Kim, M., Lee, S., Kim, D., Jeon, J., Li, D., & Yang, Y. *Why Does Self-Distillation (Sometimes) Degrade the Reasoning Capability of LLMs?*. 2026.
- [3] Kyars, K. *Reinforcement Learning from Verifiable Rewards*. 2026.